Research Intake 2026

Day 15

Named Entity Recognition & POS Tagging

A Beginner's Guide for Students

From foundational concepts to research-level understanding

Gudsky Research Foundation

Table of Contents

1. Part-of-Speech (POS) Tagging

Part-of-Speech (POS) tagging is the process of assigning a grammatical label — such as noun, verb, adjective, or preposition — to each word (token) in a sentence. It is one of the oldest and most foundational tasks in Natural Language Processing (NLP), dating back to the 1960s. Despite its apparent simplicity, POS tagging is surprisingly difficult because English (and many other languages) contains enormous ambiguity — the same word can belong to different parts of speech depending on context.

Example: She can **can** the beans. — Here 'can' is a VERB (to preserve in a tin).

Open the **can**. — Here 'can' is a NOUN (a tin container).

POS tagging resolves this ambiguity by looking at the surrounding words (context). A POS tagger processes a sentence and produces a sequence of (word, tag) pairs. For example:

Example POS-Tagged Sentence

Sentence: "The quick brown fox jumps over the lazy dog."

The → DT (Determiner)
quick → JJ (Adjective)
brown → JJ (Adjective)
fox → NN (Noun, singular)
jumps → VBZ (Verb, 3rd person singular present)
over → IN (Preposition)
the → DT (Determiner)
lazy → JJ (Adjective)
dog → NN (Noun, singular)

1.1 Tagsets: Penn Treebank and Universal Dependencies

Different NLP communities have adopted different sets of POS tags, called tagsets. The two most widely used are the Penn Treebank (PTB) tagset and the Universal Dependencies (UD) tagset.

Penn Treebank (PTB) Tagset

Developed at the University of Pennsylvania in the early 1990s, the PTB tagset contains 36 fine-grained tags for English. It is the standard for English NLP and is used by most English language models, corpora, and benchmarks.

Tag	Meaning	Example Word	Context
NN	Noun, singular	dog, city, idea	The dog barked
NNS	Noun, plural	dogs, cities	Dogs are friendly
NNP	Proper noun, singular	London, Google	Google launched...
NNPS	Proper noun, plural	Vikings, Beatles	The Beatles sang
VB	Verb, base form	run, eat, go	I run daily
VBZ	Verb, 3rd singular	runs, eats	She runs fast

VBD	Verb, past tense	ran, ate	He ran away
VBG	Verb, gerund/present participle	running, eating	He is running
JJ	Adjective	quick, tall	A quick fox
RB	Adverb	quickly, very	He ran quickly
DT	Determiner	the, a, an	The cat sat
IN	Preposition/Conjunction	in, on, over	Over the hill
CC	Coordinating conjunction	and, but, or	Fish and chips
CD	Cardinal number	one, 2, three	Two dogs ran
PRP	Personal pronoun	he, she, they	She laughed
.	Punctuation	. ! ?	End of sentence.

Universal Dependencies (UD) Tagset

UD provides a coarser 17-tag set designed to work across multiple languages consistently. It allows researchers to compare NLP systems trained on different languages using the same framework. spaCy exposes both — fine-grained PTB tags via token.tag_ and UD tags via token.pos_.

UD Tag	Meaning	Languages
NOUN	Common noun	All languages
PROPN	Proper noun	All languages
VERB	Verb	All languages
ADJ	Adjective	All languages
ADV	Adverb	All languages
DET	Determiner	Germanic, Romance
ADP	Adposition (pre/post)	All languages
CONJ / CCONJ	Conjunction	All languages
PRON	Pronoun	All languages
NUM	Numeral	All languages
PUNCT	Punctuation	All languages

1.2 Statistical POS Taggers: HMM and Maximum Entropy

Before deep learning, POS taggers relied on classical statistical models. Two approaches dominated: Hidden Markov Models (HMMs) and Maximum Entropy (MaxEnt) models. Both achieved accuracy above 96% on standard English benchmarks, making them suitable for production use.

Hidden Markov Models (HMM)

An HMM models POS tagging as a Markov chain over hidden states (POS tags) with observed outputs (words). It rests on two core assumptions:

- The Markov Assumption: The current POS tag depends only on the previous tag (bigram model). For example, an adjective (JJ) is very likely to be followed by a noun (NN).
- The Emission Assumption: The observed word depends only on its own POS tag. For example, the word 'quickly' is almost always an adverb (RB).

The HMM learns two probability tables from labelled data:

- Transition Probabilities: P(tag_i | tag_{i-1}) — how likely is each tag to follow the previous tag?
- Emission Probabilities: P(word | tag) — how likely is a word given a tag?

At inference, the Viterbi algorithm efficiently finds the most probable tag sequence in $O(N \times |T|^2)$ time, where N is sentence length and |T| is the number of tags.

> **Real-World Example: Google Search Query Parsing**
>
> Google's early search engine used HMM-based POS taggers to understand query intent. A query like 'Python books' was tagged as [NOUN NOUN] (two-noun compound → product search), while 'Python bites man' was tagged as [NOUN VERB NOUN] (subject-verb-object → news search). This tagging drove fundamentally different result rankings for the same keywords.

Maximum Entropy (MaxEnt) Taggers

MaxEnt taggers (also called logistic regression taggers) improve on HMMs by incorporating arbitrary, non-independent features of the input. Instead of being limited to the previous tag and the current word, MaxEnt can use:

- The suffix of the current word (e.g., -ing → likely VBG, -tion → likely NN)
- The prefix of the current word (pre- → often ADJ or ADV)
- Whether the word is capitalised (IsUpper → likely NNP)
- The previous two tags, not just one (trigram context)
- Whether the word contains a hyphen or digit

MaxEnt taggers achieved ~97% accuracy on Penn Treebank Wall Street Journal text, compared to ~96% for HMMs. The Stanford POS Tagger, widely used in academia throughout the 2000s and 2010s, is a MaxEnt tagger.

1.3 Neural POS Tagging with BiLSTM-CRF

Modern POS taggers are neural and learn features automatically from raw text. The dominant architecture combines a Bidirectional Long Short-Term Memory (BiLSTM) network with a Conditional Random Field (CRF) output layer.

The pipeline works as follows:

- Step 1 — Embedding: Each word is converted to a dense vector (word embedding). Pre-trained embeddings (GloVe, FastText, or contextual embeddings from BERT) are commonly used.
- Step 2 — BiLSTM Encoding: The sequence of word vectors is processed by a BiLSTM, which reads the sentence left-to-right AND right-to-left simultaneously. This gives every word access to both its left and right context.
- Step 3 — CRF Decoding: The BiLSTM output vectors are fed into a CRF layer. Unlike a simple softmax that classifies each token independently, the CRF models tag-to-tag transitions globally, enforcing constraints like 'a verb cannot directly follow a determiner in most cases.'

This architecture achieves over 98% accuracy on standard English benchmarks. spaCy, NLTK, HuggingFace Transformers, and Stanford NLP all offer implementations.

Why the CRF Layer Matters

Without CRF: The model might predict DT → DT (two determiners in a row, e.g., 'the the'), which is grammatically invalid.

With CRF: The model learns that P(DT | DT) ≈ 0 from training data and penalises such sequences globally during decoding.

This global consistency is especially important for NER, where we need B-PER → I-PER (not B-PER → I-ORG).

1.4 Applications of POS Tags

POS tags are not an end in themselves — they are a stepping stone to higher-level NLP tasks. Below are key real-world applications:

Application	How POS Tags Help	Industry Example
Named Entity Recognition	Tags narrow candidate entity spans; NNP sequences are strong NER signals	Reuters news entity extraction
Noun Phrase Chunking	DT JJ* NN+ patterns identify noun phrases for keyphrase extraction	Amazon product catalogue tagging
Sentiment Analysis	Adjectives (JJ) and adverbs (RB) carry sentiment; verbs indicate actions	Twitter brand monitoring
Machine Translation	Morphological inflection depends on POS (verb tenses differ by language)	Google Translate grammar engine
Information Extraction	VBD (past verb) signals past events; NNP signals entities	Bloomberg financial event mining
Grammar Checking	Illegal POS sequences (e.g., VB followed by VB) flag grammar errors	Microsoft Word autocorrect
Medical Record Parsing	NN clusters identify diagnoses; VB identifies procedures performed	NHS clinical NLP pipeline
Legal Document Analysis	IN chains identify contractual conditions; NNP identifies parties	Harvey AI legal review

Real-World Case Study: Bloomberg Terminal POS Tagging

Bloomberg's NLP pipeline tags every financial news article with POS tags before feeding it into downstream systems. A sentence like 'Apple raised its dividend by 5%' is tagged to identify Apple as NNP (proper noun → ORG entity), raised as VBD (past verb → action), dividend as NN (financial concept), and 5% as CD (cardinal number → quantity). This structured output powers Bloomberg's automated earnings alerts and stock movement predictions.

2. Named Entity Recognition: Foundations

Named Entity Recognition (NER) is the NLP task of locating and classifying named real-world entities mentioned in text into predefined categories such as persons, organisations, locations, dates, monetary values, and products. NER transforms raw, unstructured text into structured, machine-readable knowledge.

Example: "**Apple** [ORG] acquired **Beats Electronics** [ORG] for **$3 billion** [MONEY] in **2014** [DATE]."

NER is the entry point for a wide range of downstream tasks including knowledge graph construction, event extraction, question answering, and automated news summarisation. A NER system that works well on news articles could extract every mentioned company, person, location, and date — powering a financial intelligence dashboard with no human effort.

2.1 Entity Types and IOB/BIOES Labelling Schemes

Standard Entity Types

Most production NER systems target a core set of entity types, though domain-specific systems extend this significantly:

Entity Type	Examples	Real-World Use Case	Industry
PERSON (PER)	Barack Obama, Elon Musk, Marie Curie	Biographical databases, author disambiguation	Media, Academia
ORGANISATION (ORG)	Google, WHO, MIT, UNICEF	Corporate intelligence, citation graph building	Finance, Research
LOCATION / GPE	Paris, Amazon River, India, New York	Geospatial event mapping, travel search	Geospatial, Retail
DATE	2 March 2024, last Tuesday, Q3 2023	Timeline extraction, event ordering	Finance, Legal
TIME	3:30 PM, at midnight, two hours ago	Event scheduling, log parsing	Healthcare, Operations
MONEY	$3 billion, €500M, ₹10 crore	Financial news analysis, contract review	Finance, Legal
PRODUCT	iPhone 15, ChatGPT, Boeing 737	Product intelligence, competitive tracking	Retail, Aviation
EVENT	World Cup 2022, COVID-19 pandemic	Event tracking, news clustering	Media, Public Health
DISEASE (medical)	Diabetes Type 2, COVID-19, Alzheimer's	Clinical NLP, drug discovery	Healthcare
GENE / PROTEIN	TP53, BRCA1, mTOR pathway	Biomedical literature mining	Pharma, Research

IOB / BIOES Labelling Schemes

Because entities often span multiple words ('New York City' is a single GPE entity, not three separate entities), NER cannot simply classify each word independently. Instead, we use a labelling scheme that encodes both the entity type and the position of each token within the entity span.

IOB2 Encoding (Most Common)

- B-TYPE: Beginning of an entity span of the given type
- I-TYPE: Inside (continuation of) an entity span
- O: Outside any entity (not part of any named entity)

IOB2 Example

Sentence: "New York City is home to the United Nations."
IOB Tags: B-GPE I-GPE I-GPE O O O O B-ORG I-ORG

"Barack Obama was born in Hawaii."
B-PER I-PER O O O B-GPE

BIOES Encoding (Higher Accuracy)

BIOES extends IOB2 with two additional tags:

- S-TYPE: Single-token entity (the entire entity is one word, e.g., 'Obama' → S-PER)
- E-TYPE: End of a multi-token entity span

BIOES Example

"Barack Obama visited Paris and United Nations headquarters."
B-PER E-PER O S-GPE O B-ORG I-ORG O

Why BIOES is better: The model can distinguish S-PER (one-word person) from B-PER (start of multi-word person), improving boundary detection accuracy by 0.3–0.8 F1 on standard benchmarks.

2.2 NER as Sequence Labelling

NER is formally modelled as a structured prediction problem: given an input sequence of tokens $x = (x_1, x_2, ..., x_n)$, predict an output sequence of labels $y = (y_1, y_2, ..., y_n)$ where each y_i is an IOB or BIOES tag.

This is harder than independent per-token classification for two reasons:

- Span Consistency Constraint: An I-PER tag must follow a B-PER tag of the same type, never an O or B-ORG tag. This makes the problem globally constrained.
- Long-Range Dependencies: 'Apple' at the start of a financial article is almost certainly ORG, but 'Apple' in a recipe article is not an entity. The model must use long-range context to disambiguate.

Real-World Example: Reuters News Wire NER

Reuters processes over 1 million articles per day through its NER pipeline. Every article is tagged for PER (journalists, politicians), ORG (companies, governments), GPE (countries, cities), and MONEY (prices, valuations). These extracted entities power Thomson Reuters' Eikon financial terminal, allowing traders to instantly filter news by company or country without reading the full article.

3. Statistical NER Models

Before neural networks dominated NLP, statistical models — particularly Conditional Random Fields (CRFs) — were the state-of-the-art for NER. CRF-based systems are still used in production environments where low latency, interpretability, and low memory footprint are priorities.

3.1 Conditional Random Fields (CRF)

A CRF is a discriminative probabilistic model that directly models the conditional probability of the entire label sequence y given the observed input sequence x:

> **CRF Probability Formula**
>
> $P(y \mid x) \propto \exp(\Sigma_i \Sigma_k \lambda_k \cdot f_k(y_{i-1}, y_i, x, i))$
>
> Where:
> f_k = feature function (e.g., 'current word is capitalised AND current tag is B-PER')
> λ_k = learned weight for that feature
> y_{i-1}, y_i = previous and current label (captures transitions)
> i = position in the sequence

The key advantage of CRFs over HMMs for NER is that CRFs can incorporate arbitrary non-independent features of the input without violating any conditional independence assumptions. This is critical because NER features — capitalisation, suffix, presence in a gazetteer — are all correlated.

The Viterbi algorithm is used at inference to decode the optimal label sequence efficiently.

3.2 Feature Engineering for CRF NER

The power of a CRF NER system lies entirely in its feature set. A well-engineered feature set can push F1 from 60% to over 85% on standard benchmarks. Feature engineering is the art of encoding domain knowledge into the model.

Feature Category	Specific Features	Why It Helps
Lexical (word identity)	word.lower(), word itself	Rare proper nouns are often entities
Case / Orthographic	IsUpperCase, AllCaps, HasDigit, HasHyphen	Capitalised words → likely NNP or entity
Morphological / Suffix	word[-3:] (last 3 chars), word[-2:]	'-son' → person name, '-Corp' → org
Morphological / Prefix	word[:3] (first 3 chars)	Pre-, Bio- suggest certain entity types
POS Tag	Current and surrounding POS tags	NNP clusters are strong entity signals
Context Window	±2 words and their features	Surrounding words disambiguate entity type
Gazetteer Membership	InPersonList, InCompanyList, InCityList	Known entity lists boost recall dramatically

Position in sentence	IsFirstToken, IsLastToken	Named entities often appear at sentence start
Label Transitions	P(I-ORG | B-ORG) vs P(I-PER | B-ORG)	Enforces IOB consistency constraints

Impact of Gazetteers on CRF NER

Benchmark: CoNLL-2003 English NER dataset (news articles from Reuters).
CRF without gazetteers: F1 = 75.2%
CRF with company + person gazetteers: F1 = 82.6% (+7.4 points)
CRF with all gazetteers (cities, countries, organisations, persons): F1 = 84.9%

Gazetteers are lists of known entity names (e.g., all Fortune 500 companies, all world capital cities). They are the single highest-impact feature for CRF NER in news domains.

Real-World Case Study: CRF NER in Legal Tech (Kira Systems)

Kira Systems (now Litera) uses CRF-based NER to extract party names, dates, monetary amounts, and obligation clauses from legal contracts. CRF was preferred over neural models for three reasons: (1) Legal teams can inspect feature weights to understand why an entity was extracted (interpretability). (2) CRF runs in under 10ms per contract page (low latency). (3) Lawyers can add domain-specific gazetteers (law firm names, jurisdiction lists) without retraining the model.

4. Neural NER: BiLSTM-CRF and Transformers

Neural NER models eliminated the need for manual feature engineering by learning representations directly from raw text. The two dominant architectures are BiLSTM-CRF (for sequences) and Transformer-based models (BERT and its variants).

4.1 BiLSTM Character + Word Embeddings

The landmark paper by Ma & Hovy (2016) — 'End-to-end Sequence Labeling via Bi-directional LSTM-CNNs-CRF' — achieved state-of-the-art NER without any hand-crafted features. The architecture combines three components:

- Character-Level CNN: A Convolutional Neural Network reads individual characters of each word and produces a character embedding. This captures morphological patterns — 'Obama' and 'Obama's' share character-level structure, and '-tion' endings reliably signal nouns.
- Word-Level Pre-trained Embeddings: GloVe or Word2Vec embeddings provide semantic word vectors. Words with similar meanings (e.g., 'Apple', 'Google', 'Microsoft') cluster nearby in embedding space.
- BiLSTM Encoder: Concatenated character + word embeddings are fed into a BiLSTM. The forward LSTM captures left context; the backward LSTM captures right context. Each token gets a rich contextual representation.
- CRF Output Layer: Enforces globally consistent IOB label sequences across the sentence.

Performance Milestone

Ma & Hovy (2016) on CoNLL-2003 English NER:
F1 = 91.21% — surpassing a decade of manual CRF feature engineering
No task-specific features used — purely learned representations
This result demonstrated that neural models could match or exceed expert human feature engineering in NER.

4.2 BERT for NER: Token Classification Head

BERT (Bidirectional Encoder Representations from Transformers, Devlin et al., 2018) revolutionised NER by providing deep bidirectional contextual representations pre-trained on 3.3 billion words of text.

Fine-tuning BERT for NER is remarkably simple architecturally: a linear classification head is added on top of BERT's final token representations:

BERT NER Architecture

Input: [CLS] Apple acquired Beats for $3B in 2014 [SEP]
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
BERT: Contextual token embeddings (768 or 1024 dimensions each)
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
Linear: h_i → softmax(W × h_i + b) → IOB label probability distribution
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
Output: O B-ORG O B-ORG O B-MONEY O B-DATE

Why BERT is so powerful for NER: BERT's attention mechanism allows every token to attend to every other token in the sentence. This resolves long-range ambiguity — 'Apple' in a tech article attends to surrounding words like 'acquired', 'stock', 'CEO', which strongly signal ORG rather than FOOD.

Model	CoNLL-2003 English NER F1
CRF with manual features (2005)	~84%
BiLSTM-CRF, Ma & Hovy (2016)	91.21%
BERT-base fine-tuned (2019)	92.4%
BERT-large fine-tuned (2019)	92.8%
BERT-large + CRF (2019)	93.5%
SpanBERT (2020)	93.9%
Human performance estimate	~97%

4.3 Span-Based NER

Standard IOB-based NER cannot represent nested entities — entities that contain other entities within them. Consider:

Nested Entity Example

"The [University [of [California] GPE] ORG] ORG Berkeley team won the award."

Here: 'California' is a GPE (geographic political entity)
'University of California' is an ORG (organisation)
'University of California Berkeley' is also an ORG (a different organisation)

IOB encoding can only assign one label per token — it cannot express that 'California' is simultaneously INSIDE an ORG AND is itself a GPE.

Span-based NER models solve this by enumerating all possible token spans up to a maximum length L and classifying each span independently as either an entity type or non-entity.

- Every pair (i, j) where $j - i \leq L$ is a candidate span
- A span encoder (typically BERT) produces a representation for each span
- A classifier assigns each span a label (PER, ORG, GPE, ..., or O)

Real-World Example: Biomedical Nested NER (PubMed)

In biomedical literature, nested entities are extremely common: '[Interleukin-6 [receptor]] antagonist' contains both a GENE entity ('Interleukin-6') and a PROTEIN entity ('Interleukin-6 receptor'). Pharmaceutical companies like Pfizer and Roche use span-based NER (SpanBERT) on PubMed abstracts to extract drug-disease-gene relationships for drug discovery pipelines, where standard IOB NER would miss nested mentions and lose critical biological information.

5. spaCy NER Pipeline In Depth

spaCy is an industrial-strength, open-source NLP library for Python, developed by Explosion AI. It is widely used in production NLP systems globally because it is fast, accurate, and provides a complete end-to-end pipeline. Understanding spaCy's architecture helps us understand modern NER systems in general.

5.1 spaCy's Architecture

spaCy processes text through a pipeline of components, each performing a specific NLP task. The default English pipeline includes:

Pipeline Component	Function
tokenizer	Splits raw text into tokens (words, punctuation). Rule-based, no model needed.
tok2vec	Converts tokens to contextual vectors using CNN/Transformer. Shared across components.
tagger	Assigns POS tags (both UD coarse tags and PTB fine tags) to each token.
parser	Performs dependency parsing — assigns syntactic head and relation to each token.
ner	Named entity recognition — identifies and classifies entity spans.
lemmatizer	Reduces words to their base form (running → run, mice → mouse).
attribute_ruler	Applies rule-based overrides using patterns (e.g., always tag 'iPhone' as PRODUCT).

5.2 spaCy's NER Component

spaCy's NER component is a transition-based entity recogniser with a CNN encoder. Unlike LSTM-based models, it processes the document in a single left-to-right pass using a set of actions (SHIFT, REDUCE, BEGIN, IN, LAST, UNIT) to build entity spans incrementally.

Key properties of spaCy's NER:

- Speed: 2–5× faster than BiLSTM-CRF because it avoids recurrent computation. Can process 10,000–50,000 words per second on CPU.
- Accuracy: The en_core_web_trf model (using RoBERTa transformer backbone) achieves over 90 F1 on OntoNotes 5.0.
- Pipeline integration: NER runs after POS tagging, so it can use POS tag features to improve entity detection.

spaCy Models Available for English

en_core_web_sm — Small (12 MB). Fast, lower accuracy. Suitable for prototyping.
en_core_web_md — Medium (40 MB). Includes word vectors. Good for most tasks.
en_core_web_lg — Large (560 MB). Better word vectors. High accuracy on standard text.
en_core_web_trf — Transformer-based (500 MB). Highest accuracy. Uses RoBERTa backbone.

Note: Coding implementation (loading models, running the pipeline, visualising results with displacy) will be covered in a dedicated coding session using Python/Jupyter Notebook.

5.3 Available NLP Libraries for NER and POS Tagging

Multiple production-grade libraries implement NER and POS tagging. You will explore these in the coding sessions:

Library	Best For	NER Approach	POS Tagging
spaCy	Production, speed, pipelines	Transition-based CNN / Transformer	☑ Fine + Coarse tags
NLTK	Teaching, research prototyping	MaxEnt tagger + chunking	☑ Penn Treebank
HuggingFace Transformers	State-of-the-art accuracy, research	BERT / RoBERTa token classification	☑ Via fine-tuning
Stanford CoreNLP	Multi-language, academic research	CRF + Neural	☑ Penn Treebank
Flair	Contextual string embeddings	BiLSTM-CRF with Flair embeddings	☑ Multi-language
stanza	Multi-language support	BiLSTM-CRF (Stanford NLP)	☑ UD tagset

6. Dependency Parsing

Dependency parsing is the task of analysing the grammatical structure of a sentence by establishing relationships between words. Specifically, it assigns a directed dependency arc between each word (the dependent) and its syntactic head (the word it depends on), and labels each arc with the grammatical relation.

Unlike constituency parsing (which builds phrase trees: NP, VP, PP), dependency parsing directly links words to words. This makes it more compact, language-agnostic, and directly useful for information extraction.

6.1 Dependency Relations

Every sentence's dependency parse consists of arcs labelled with grammatical relations. The most common relations:

Relation	Meaning	Example
nsubj	Nominal subject — who/what does the verb	Apple [nsubj] → acquired
dobj	Direct object — who/what receives the action	Beats [dobj] → acquired
amod	Adjectival modifier — adjective modifying noun	quick [amod] → fox
prep	Prepositional modifier	for [prep] → acquired
pobj	Object of a preposition	$3B [pobj] → for
det	Determiner — article modifying noun	The [det] → fox
advmod	Adverbial modifier	quickly [advmod] → ran
compound	Compound noun modifier	New [compound] → York
ROOT	Root of the sentence — main verb	acquired [ROOT]
conj	Conjunct — connected by coordinating conjunction	Google [conj] → Apple

Dependency Parse Example

Sentence: "Apple acquired Beats for $3 billion in 2014."

```
acquired [ROOT]
  ├── Apple [nsubj]   → Apple is the subject of 'acquired'
  ├── Beats [dobj]    → Beats is the direct object of 'acquired'
  ├── for [prep]      → 'for' is a prepositional modifier of 'acquired'
  │     └── billion [pobj]  → 'billion' is the object of 'for'
  │           └── $3 [quantmod]  → '$3' modifies 'billion'
  └── in [prep]       → 'in' is a prepositional modifier of 'acquired'
        └── 2014 [pobj]   → '2014' is the object of 'in'
```

6.2 Transition-Based vs. Graph-Based Parsing

Two major approaches to dependency parsing exist:

Transition-Based Parsing (Arc-Eager / Arc-Standard)

The parser maintains a stack of partially processed words and a buffer of remaining input words. At each step, it applies one of four actions:

- SHIFT: Move the next word from buffer onto the stack
- LEFT-ARC: Create a dependency arc from the top of stack to the second-top; remove second-top
- RIGHT-ARC: Create a dependency arc from second-top to top of stack; move top to output
- REDUCE: Pop the top of stack (it is fully processed)

Transition-based parsers are very fast (linear time: O(N)) but can struggle with non-projective dependencies (crossing arcs) common in free-word-order languages like German, Czech, or Hindi.

Graph-Based Parsing

Graph-based parsers score all possible dependency arcs globally and find the maximum spanning tree (MST) over the complete directed graph. They handle non-projective dependencies naturally but are slower ($O(N^2)$ or $O(N^3)$).

Real-World Example: Subject-Verb-Object Extraction for Knowledge Graphs

Google's Knowledge Graph ingests millions of Wikipedia and news sentences daily. Dependency parsing extracts Subject-Verb-Object (SVO) triples: nsubj arc gives the subject, dobj arc gives the object, ROOT gives the verb. From 'Elon Musk founded SpaceX in 2002', the SVO triple ('Elon Musk', 'founded', 'SpaceX') is extracted and added to the knowledge graph, enabling Google to answer 'Who founded SpaceX?' without storing the full sentence.

6.3 Dependency Parsing in spaCy

spaCy uses a transition-based dependency parser (arc-eager system) with a neural CNN encoder. Every token in a parsed document gets three attributes:

- token.dep_ — the dependency relation label (e.g., 'nsubj', 'dobj', 'amod')
- token.head — the syntactic head token (the word this token depends on)
- token.children — the list of tokens that depend on this token

Downstream applications using spaCy dependency parsing include: relation extraction (What did who do to whom?), coreference resolution preparation, and semantic role labelling.

Real-World Example: Relation Extraction at LinkedIn

LinkedIn uses dependency parsing to extract professional relationships from job descriptions and user bios. From the sentence 'She managed a team of 50 engineers at Microsoft', the parser identifies: subject 'She' (nsubj → managed), object 'team' (dobj → managed), and modifier 'at Microsoft' (prep → managed, pobj → Microsoft). This powers LinkedIn's search for 'people who managed engineering teams at Microsoft.'

7. Evaluation: Span-Level F1

Evaluating NER correctly is critical — using the wrong metric can give a misleading picture of system quality. The standard evaluation metric for NER is span-level F1, also called entity-level F1.

7.1 Why Token-Level Accuracy Fails for NER

Naively, one might evaluate NER by computing accuracy at the token level: what fraction of tokens were correctly labelled? This metric is deeply misleading for two reasons:

- Class Imbalance: In a typical news article, over 80% of tokens are labelled O (outside any entity). A model that always predicts O would achieve 80%+ token accuracy while being completely useless for NER.
- Partial Match Illusion: If the true entity is 'New York City' (B-GPE I-GPE I-GPE) and the model predicts 'New York' (B-GPE I-GPE O), token accuracy gives credit for 2 out of 3 tokens. But this is a wrong entity extraction — the boundary is incorrect.

7.2 Span-Level F1 Definition

Span-level F1 requires a predicted entity to be EXACTLY correct in both span boundaries AND entity type to count as a true positive. There is no partial credit.

Span-Level F1 Formulas

Precision = True Positives / (True Positives + False Positives)
= (Number of correctly predicted entities) / (Total predicted entities)

Recall = True Positives / (True Positives + False Negatives)
= (Number of correctly predicted entities) / (Total gold entities)

F1 Score = 2 × (Precision × Recall) / (Precision + Recall)
= Harmonic mean of Precision and Recall

7.3 Worked Example

Consider a sentence with three gold (true) entities and a system's predictions:

Entity	Gold Label	Predicted Label	Correct?
Apple	ORG	ORG	☑ True Positive
New York City	GPE	GPE (but boundary: 'New York')	✗ False Negative (wrong span)
$3 billion	MONEY	MONEY	☑ True Positive
Tim Cook	PER	(not predicted)	✗ False Negative (missed)
2014	(not an entity)	DATE	✗ False Positive (hallucinated)

From this example:

- True Positives (TP) = 2 (Apple/ORG and $3B/MONEY both exactly correct)
- False Positives (FP) = 1 (2014 predicted as DATE but not in gold standard)
- False Negatives (FN) = 2 (New York City missed due to wrong boundary; Tim Cook missed entirely)

F1 Calculation

Precision = TP / (TP + FP) = 2 / (2 + 1) = 0.667
Recall = TP / (TP + FN) = 2 / (2 + 2) = 0.500
F1 = 2 × (0.667 × 0.500) / (0.667 + 0.500) = 0.572

Despite getting 2 out of 4 true entities correct, F1 is only 57.2% — the strict metric rightfully penalises boundary errors and hallucinations.

7.4 CoNLL Evaluation Script

The standard tool for NER evaluation is the CoNLL-2003 evaluation script (conlleval). It computes span-level precision, recall, and F1 per entity type AND overall. It is the benchmark tool used in all major NER papers and competitions. When comparing NER systems, always ensure you are using span-level F1 with the CoNLL script — many papers report different metrics, making direct comparison difficult.

Real-World Example: NER Evaluation at Google DeepMind Health

When Google DeepMind built an NER system for NHS clinical notes to extract diagnoses and medications, token-level accuracy reported 94%. However, span-level F1 was only 78% — because the model frequently extracted 'Type 2' instead of 'Type 2 Diabetes' (wrong span), and missed multi-word drug names like 'amoxicillin clavulanate' (predicting only 'amoxicillin'). The strict span-level metric correctly identified that the model needed improvement on multi-token medical entities.

8. Domain Adaptation for NER

NER models trained on one domain frequently perform poorly when applied to a different domain. This is called domain shift — the statistical patterns the model learned (vocabulary, entity surface forms, writing style) do not match the target domain. Understanding domain adaptation is essential for any real-world NER deployment.

8.1 The Domain Shift Problem

Source Domain	Target Domain	F1 Drop	Primary Cause
News (CoNLL-2003)	Biomedical (JNLPBA)	92% → 51%	New entity types (GENE, PROTEIN, CELL_LINE), different vocabulary
News (CoNLL-2003)	Legal contracts (LegalNER)	92% → 58%	Legal party names, clause references, dates in different formats
News (CoNLL-2003)	Scientific papers (SciERC)	92% → 46%	Technical entity types (METHOD, TASK, METRIC), acronyms
News (CoNLL-2003)	Social media (Twitter NER)	92% → 62%	Spelling variations, hashtags, no capitalisation, slang

8.2 Domain Adaptation Strategies

Strategy 1: Domain-Specific Pre-training

The most effective strategy is to pre-train a language model on the target domain's text before fine-tuning for NER. This gives the model domain-appropriate vocabulary representations.

Domain-Specific BERT Variants

BioBERT — Pre-trained on PubMed abstracts + PMC full articles (18B words). Achieves 10+ F1 improvement on biomedical NER over standard BERT.

LegalBERT — Pre-trained on EU legislation, court cases, and legal contracts. Outperforms BERT on legal NER by 6-9 F1 points.

FinBERT — Pre-trained on financial news and earnings call transcripts. Better entity disambiguation for financial NER (company names, tickers).

SciBERT — Pre-trained on computer science and biomedical papers from Semantic Scholar. Better for scientific NER tasks.

Strategy 2: Few-Shot Learning with Prototypical Networks

When labelled data in the target domain is scarce (5–50 examples per entity type), few-shot learning methods can adapt a general NER model with minimal annotation effort.

Prototypical networks work by computing a prototype vector for each entity type (the average embedding of all example entities of that type), and classifying new spans by finding the nearest prototype in embedding space. This requires no gradient updates — just a handful of examples.

Real-World Example: Few-Shot NER for Rare Disease Detection (Rare Diseases UK)

Rare Diseases UK needed a NER system to extract mentions of rare disease names from patient forums. Only 20–30 labelled examples were available per disease. Using prototypical network few-shot NER with BioBERT embeddings, they achieved 71% F1 on rare disease NER with just 15 labelled examples per disease — compared to 38% F1 from a standard fine-tuned BERT needing 500+ examples per type.

Strategy 3: Active Learning

Active learning is an iterative annotation strategy where the model selects the most informative unlabelled examples for human annotation. Instead of labelling all available text (expensive), the annotator only labels the sentences the model is most uncertain about.

The active learning loop:

- Train initial NER model on small seed dataset
- Run model on large unlabelled pool
- Rank sentences by model uncertainty (e.g., low prediction confidence, high entropy)
- Present top-K most uncertain sentences to human annotator
- Add newly labelled sentences to training set and retrain
- Repeat until performance plateaus

Active Learning Efficiency

Domain: Clinical NER (medical records)
Full dataset annotation cost: 2,000 hours
Active learning with 10 iterations: 200 hours of annotation
Resulting F1: 89% (vs 91% from full annotation) — 90% cost saving for 2% F1 loss.
Source: Settles (2012) — 'Active Learning Literature Survey'

Real-World Case Study: NHS Clinical NLP at Scale

NHS England deployed a NER system to extract diagnoses, medications, and procedures from 50+ million free-text clinical notes stored in the NHS Digital infrastructure. The challenge: clinical notes were written across 50+ hospitals with wildly varying writing styles, abbreviations, and local terminology. Solution: (1) ClinicalBERT pre-trained on MIMIC-III clinical notes, (2) Active learning to annotate 3,000 high-uncertainty notes per hospital, (3) BNF (British National Formulary) drug name gazetteer integrated as a feature. Final system: 88% F1 on drug names, 83% F1 on diagnoses — processing 1 million notes per day.

9. Research Frontiers

NER and POS tagging remain active research areas. While standard news-domain NER is considered largely solved (approaching human performance), several frontier challenges attract significant research attention.

9.1 Nested Named Entity Recognition

Standard IOB NER assumes entities are flat and non-overlapping. Real text — especially in scientific and legal domains — contains nested entities where one entity span contains another.

Nested NER Examples

Legal: '[The Government [of [France]_GPE]_ORG]_ORG announced...'
→ France is a GPE; 'Government of France' is an ORG; they overlap.

Biomedical: 'A mutation in the [BRCA1 [gene]_GENE_COMPONENT]_GENE was detected.'
→ 'BRCA1' is a specific gene; 'gene' is a generic biological component.

Corporate: '[Apple [Music]_PRODUCT]_ORG launched in India.'
→ 'Music' is a product; 'Apple Music' is also a product with a different meaning.

Approaches to nested NER include:

- Span-based models (SpanBERT) — enumerate all spans and classify independently
- Layered sequence labelling — run multiple IOB passes, each capturing entities at a different nesting depth
- Hypergraph-based models — represent overlapping entity spans as a hypergraph

Benchmark: ACE 2005 Nested NER

SpanBERT: 85.3 F1 on nested NER
Layered BiLSTM-CRF: 79.2 F1
Flat IOB BERT (cannot represent nesting): effectively 0% F1 for nested mentions

9.2 Cross-Lingual NER

Cross-lingual NER addresses the challenge of low-resource languages where labelled NER data is scarce or non-existent. The goal is to train on high-resource languages (English, German, French) and transfer to low-resource languages (Swahili, Tagalog, Yoruba) without target-language labelled data.

Key approaches:

- Zero-shot cross-lingual transfer: Fine-tune on English NER, evaluate directly on Hindi or Swahili without any target-language training data. XLM-RoBERTa achieves 50–65 F1 on zero-shot NER across 40 languages.
- Cross-lingual data augmentation: Machine-translate English NER training data into the target language, preserving entity spans via alignment.

- Multilingual fine-tuning: Fine-tune on labelled data from multiple languages simultaneously, sharing representations across languages.

Real-World Example: UN Multilingual News Monitoring

The United Nations monitors news in 193 member states across dozens of languages for early warning of conflicts, humanitarian crises, and human rights violations. Using XLM-RoBERTa fine-tuned on English and French NER then zero-shot transferred to Amharic, Somali, and Tigrinya, the system extracts PER, ORG, and GPE entities from African regional news with 58–65% F1 — sufficient for conflict monitoring dashboards without requiring annotators fluent in each target language.

9.3 Document-Level NER

Most NER models process one sentence at a time, missing critical cross-sentence context. Consider: 'Johnson won the election. He was congratulated by Macron.' — A sentence-level model might not know 'He' refers to Johnson unless it sees the previous sentence. Document-level NER maintains entity memory across sentences.

Approaches to document-level NER:

- Entity span memory: Luan et al. (2019) maintain a dynamic entity mention graph across sentences, enabling later mentions to resolve earlier ambiguous mentions.
- Long-context transformers: Longformer and BigBird extend BERT-style attention to documents of 4,096–16,384 tokens, enabling full-document context for NER.
- Coreference-enhanced NER: First resolve coreferences (he → Obama, the company → Apple), then run NER on the resolved text.

Research Result: Document-Level NER on SCIERC

SCIERC dataset: scientific paper NER with long-range entity dependencies.

Sentence-level BERT NER: F1 = 67.2%

Document-level NER with entity memory (DyGIE++): F1 = 73.6%

Improvement: +6.4 F1 — attributable entirely to cross-sentence entity context.

Most of the improvement comes from correctly labelling entities that appear only once but are disambiguated by later mentions.

9.4 LLM-Based NER: GPT and Claude for Entity Extraction

Large Language Models (LLMs) like GPT-4 and Claude can perform NER zero-shot — without any fine-tuning — by providing a natural language prompt describing the entity types and asking the model to extract entities from the text. This approach is increasingly used in industry for rapid prototyping and for novel entity types where no labelled training data exists.

Advantages of LLM-based NER:

- Zero labelled data required — define entity types in natural language
- Handles novel entity types and nuanced definitions effortlessly
- Can reason about ambiguous entities and explain its decisions

Disadvantages:

- Higher latency and cost than specialised NER models

- F1 on standard benchmarks (CoNLL-2003) is typically 85–88% — below fine-tuned BERT (93%)
- Output format less consistent — requires structured output prompting

Real-World Example: Harvey AI Legal NER with LLMs

Harvey AI (legal tech startup) uses GPT-4 for NER on novel legal entity types (e.g., 'obligations under Section 7', 'material adverse change conditions') that no traditional NER dataset covers. Their approach: prompt GPT-4 with a legal ontology definition, extract entities as JSON, and post-process. For standard entity types (party names, dates), they use fine-tuned BERT-based NER for speed and reliability, falling back to GPT-4 only for rare or complex entity types.

A. Assignment: NER System for News Articles

This graded assignment requires you to design and evaluate a complete Named Entity Recognition pipeline for BBC or Reuters news articles. The assignment tests your understanding of all concepts covered in today's session: entity types, IOB labelling, model architecture selection, span-level F1 evaluation, and error analysis.

A.1 Assignment Overview

Parameter	Details
Dataset	BBC News articles (technology, business, and politics sections) OR Reuters CoNLL-2003 test set
Target Entity Types	PERSON, ORGANISATION, GPE (Geopolitical Entity), DATE, MONEY
Minimum Requirement	spaCy baseline with en_core_web_sm — achieve ≥ 82 F1
Stretch Goal	Fine-tuned BERT token classifier — achieve ≥ 90 F1
Evaluation Metric	Span-level F1 (strict CoNLL evaluation — both span and type must be correct)
Submission	Python notebook (.ipynb) + written report (PDF) + demonstration on 10 unseen articles

A.2 System Requirements

Your NER system must implement the following five components:

- Component 1 — Data Loading: Load raw article text from files or an API. Handle encoding issues (UTF-8), remove HTML tags if present, and segment text into sentences.
- Component 2 — NER Pipeline Application: Apply at minimum the spaCy NER pipeline to each sentence. Extract all entity spans with their type labels.
- Component 3 — Optional BERT Fine-tuning: For the stretch goal, fine-tune a BERT token classification head on the CoNLL-2003 training set. Report both spaCy baseline and BERT results.
- Component 4 — Evaluation: Compute span-level precision, recall, and F1 per entity type AND overall on a held-out test set. Present results in a formatted table.
- Component 5 — Error Analysis and Visualisation: Identify and discuss the top 3 error categories your system makes (e.g., wrong entity type, boundary errors, hallucinations). Visualise entity distributions using a bar chart or pie chart.

A.3 Evaluation Rubric

Component	Marks	Criteria
spaCy baseline running correctly	20	Pipeline loads, processes all articles, outputs IOB labels
Span-level F1 ≥ 82% (baseline)	20	CoNLL evaluation script used; results table per entity type

Error analysis (3+ error categories)	20	Specific examples shown; root cause identified for each
Visualisation of entity distributions	10	Bar chart or pie chart; correctly labelled with counts
BERT fine-tuning (stretch goal)	15	Training loss plotted; F1 ≥ 90% on test set
Written report quality	15	Clear explanation of architecture choices and findings

A.4 Hints and Guidance

Key Hints for the Assignment

1. Tokenisation matters: spaCy and BERT tokenise differently. BERT uses WordPiece (splits 'playing' → 'play' + '##ing'). When converting BERT token predictions back to word-level IOB labels, only use the prediction for the FIRST subword of each word.

2. Evaluation tools: Use the seqeval Python library for span-level F1 — it implements the CoNLL evaluation standard and handles IOB/BIOES formats. Available libraries: spaCy, NLTK, HuggingFace Transformers, seqeval — all to be explored in coding sessions.

3. Common error types to look for: (a) Boundary errors — correct type but wrong span (partial match), (b) Type confusion — correct span but wrong entity type (ORG predicted as GPE), (c) False negatives — entities missed entirely, (d) False positives — non-entities labelled as entities.

4. Baseline first: Always establish the spaCy baseline before attempting BERT fine-tuning. This gives you a performance floor and lets you quantify the improvement from fine-tuning.

Reference Papers for Further Reading

Lample et al. (2016) — 'Neural Architectures for Named Entity Recognition' (BiLSTM-CRF)
Ma & Hovy (2016) — 'End-to-end Sequence Labeling via Bi-directional LSTM-CNNs-CRF'
Devlin et al. (2018) — 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding'
Joshi et al. (2019) — 'SpanBERT: Improving Pre-training by Representing and Predicting Spans'
Conneau et al. (2020) — 'Unsupervised Cross-lingual Representation Learning at Scale' (XLM-RoBERTa)
Settles (2012) — 'Active Learning Literature Survey' (Chapter 3 for NER active learning)

The future of AI is bright, and you can be part of it!